Desafío 2: Fiesta mundial con UdeAWorldCup

Juan Diego Arias Toro

1192792031

Informática 2

2026

1. Análisis del problema y consideraciones para la alternativa de solución propuesta

El desafío consiste en desarrollar un sistema en C++ capaz de simular un torneo mundial de fútbol con 48 selecciones nacionales. El programa debe partir de información histórica almacenada en un archivo CSV, construir los bombos de sorteo, conformar los grupos bajo restricciones deportivas, calendarizar la fase de grupos, simular todos los partidos del torneo, actualizar estadísticas históricas de equipos y jugadores, generar las fases eliminatorias hasta la final y producir un reporte final con los resultados más relevantes del campeonato.

El problema no se reduce únicamente a simular marcadores aleatorios. La dificultad principal está en integrar varias restricciones simultáneas: los grupos deben respetar reglas por confederación, el calendario debe cumplir límites de partidos por día y descanso mínimo entre equipos, los empates de fase de grupos deben afectar la clasificación, los empates de eliminación directa deben resolverse mediante un criterio adicional, y los datos generados deben mantenerse consistentes tanto en memoria como en archivos de salida.

Para evitar una solución trivial, la alternativa propuesta se basa en una arquitectura orientada a objetos. Cada entidad importante del torneo se modela mediante una clase: `Equipo`, `Jugador`, `Partido`, `Grupo`, `Torneo`, `Utilidades` y `MedidorRecursos`. Esta separación permite asignar responsabilidades claras a cada módulo y evita concentrar toda la lógica en el `main.cpp`.

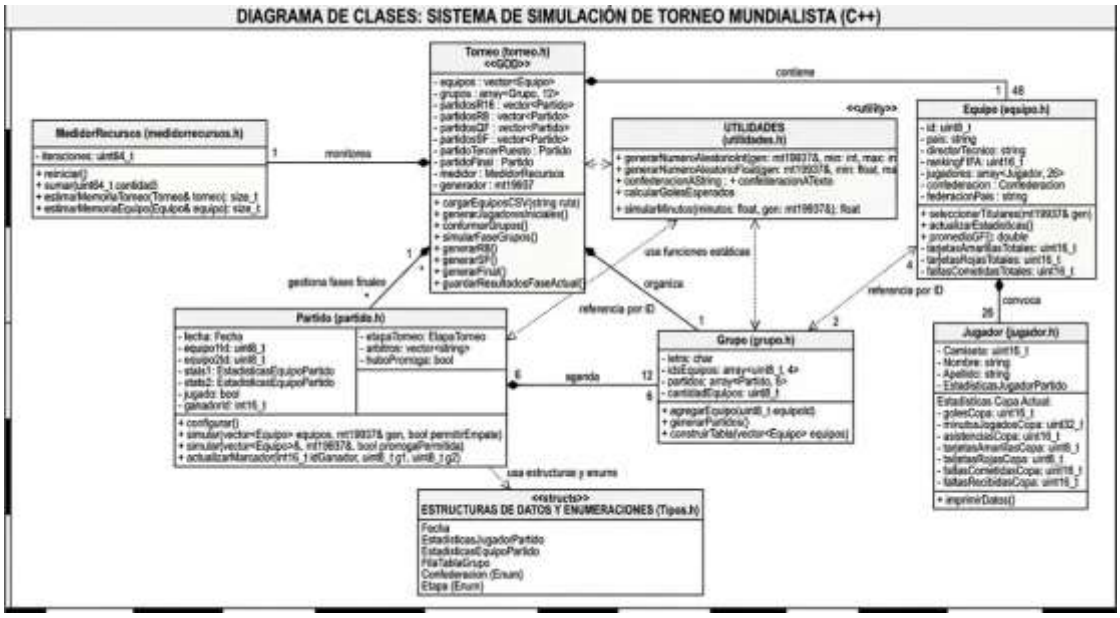
La clase `Torneo` actúa como coordinadora general del sistema. Se encarga de cargar los equipos desde el archivo CSV, crear los bombos, conformar los grupos, generar el calendario, ejecutar las simulaciones, construir las fases eliminatorias, emitir reportes y guardar los datos actualizados. La clase `Grupo` organiza los cuatro equipos de cada grupo y genera sus partidos internos. La clase `Partido` concentra la simulación de un encuentro, incluyendo goles esperados, posesión, titulares, estadísticas individuales y actualización de históricos. La clase `Equipo` almacena los datos generales de cada selección y administra sus jugadores, mientras que `Jugador` guarda tanto estadísticas históricas como estadísticas específicas de la copa actual.

Una consideración importante fue el uso controlado de memoria. En lugar de crear objetos dinámicos innecesarios con `new` y `delete`, se utilizaron estructuras de tamaño conocido cuando era posible, como `std::array` para grupos, jugadores titulares, árbitros y jugadores de cada equipo. Para colecciones variables, como bombos o partidos de fases eliminatorias, se utilizaron `std::vector`, procurando reservar capacidad cuando era conveniente. Esta decisión reduce errores de memoria, mejora la claridad del programa y mantiene una administración eficiente de recursos.

También se implementó una medición básica de recursos mediante la clase `MedidorRecursos`, que permite registrar iteraciones y estimar la memoria usada por las estructuras principales del sistema. Aunque esta medición no equivale a un perfilador real de bajo nivel, sí permite documentar el comportamiento computacional de cada fase del programa.

2. Diagrama de clases de la solución planteada

A continuación se presenta el diagrama de clases



La relación principal del diseño es jerárquica: el `Torneo` contiene los equipos, grupos y partidos de fases eliminatorias. Cada `Grupo` contiene los identificadores de sus cuatro equipos y sus seis partidos. Cada `Partido` opera sobre dos equipos y actualiza sus estadísticas. Cada `Equipo` contiene sus 26 jugadores. Las clases `Utilidades` y `MedidorRecursos` apoyan el funcionamiento general sin representar entidades deportivas.

3. Algoritmos esenciales debidamente intra-documentados

En esta sección se describen en alto nivel los subprogramas cuya solución no es trivial. No se copia código fuente; se explica la lógica usada para resolver cada tarea.

3.1 Carga de equipos desde CSV

La carga inicial consiste en abrir el archivo `selecciones_clasificadas_mundial.csv`, leer sus líneas y separar cada registro por el delimitador definido. Para cada fila válida se extraen los datos principales de la selección: ranking FIFA, país, director técnico, federación, confederación, goles históricos, goles recibidos y resultados históricos. Con estos datos se construye un objeto `Equipo`.

La función también convierte el texto de la confederación a un valor enumerado interno. Esto permite que el resto del programa trabaje con valores controlados, como `UEFA`, `CONMEBOL`, `CONCACAF`, `CAF`, `AFC`, `OFC` o `DESCONOCIDA`, en lugar de comparar cadenas de texto constantemente.

Después de cargar los equipos, el programa genera artificialmente 26 jugadores por selección. Cada jugador recibe un número de camiseta, nombre y apellido genéricos, y sus estadísticas iniciales comienzan en cero. Esta decisión permite simular estadísticas individuales aunque el archivo original solo contenga información agregada por equipo.

3.2 Creación de bombos

La creación de bombos se basa principalmente en el ranking FIFA. Primero se identifica al anfitrión, `United States`, y se ubica en el primer bombo. Luego se ordenan los demás equipos según su ranking y se reparten en cuatro bombos de 12 equipos.

La lógica busca respetar la estructura de sorteo del torneo: los mejores equipos se distribuyen en bombos superiores y los equipos restantes se asignan progresivamente. Esto evita una distribución completamente aleatoria e incorpora un criterio deportivo previo a la conformación de grupos.

3.3 Conformación de grupos mediante backtracking

La conformación de grupos es una de las partes más importantes del proyecto. La solución trivial sería asignar equipos secuencialmente o de manera completamente aleatoria; sin embargo, eso puede romper restricciones de confederación. Por esta razón se utilizó backtracking.

El procedimiento toma los equipos de los bombos y trata de ubicarlos en los 12 grupos. Antes de agregar un equipo a un grupo, verifica que el grupo no esté lleno y que la confederación del equipo sea compatible con las restricciones actuales. Si el equipo puede agregarse, se inserta temporalmente y el algoritmo continúa con el siguiente equipo. Si más adelante se llega a una configuración imposible, el algoritmo retrocede, retira el último equipo agregado y prueba otra alternativa.

Este enfoque permite buscar una distribución válida sin asumir que la primera asignación posible será correcta. Aunque el backtracking puede ser costoso en problemas grandes, aquí el tamaño del problema es controlado: 48 equipos, 12 grupos y 4 bombos. Además, se organiza la asignación por rondas de bombo, lo que reduce combinaciones innecesarias.

3.4 Generación del calendario de grupos

Inicialmente se consideró una estrategia greedy para ubicar partidos según disponibilidad de fechas. Sin embargo, esa solución podía producir una salida desordenada y dependía demasiado del orden interno de los partidos. La versión final usa una solución determinista por jornadas.

Cada grupo de cuatro equipos tiene seis partidos, organizados en tres jornadas:

- Jornada 1: dos partidos.
- Jornada 2: dos partidos.
- Jornada 3: dos partidos.

Luego, los grupos se programan por pares. En cada día se juegan dos partidos de un grupo y dos partidos de otro grupo, para un total de cuatro partidos diarios. Las jornadas de cada par de grupos se separan por varios días, garantizando que ningún equipo juegue de nuevo antes de cumplir el descanso mínimo requerido.

Esta estrategia es más eficiente que la versión greedy porque no necesita buscar posibilidades día a día. Además, es fácil de justificar: siempre respeta el máximo de cuatro partidos por día, distribuye los encuentros de forma ordenada y garantiza el descanso entre partidos.

3.5 Simulación de un partido

La simulación de un partido se basa en varias etapas. Primero se calculan los goles esperados de cada equipo mediante una fórmula que combina la capacidad ofensiva histórica del equipo atacante y la debilidad defensiva histórica del rival. La fórmula utiliza los parámetros `mu`, `alpha` y `beta`, y produce un valor esperado `lambda`.

Luego, ese valor esperado se convierte en goles reales usando una distribución probabilística. Esto permite que los resultados no sean deterministas, pero sí estén influenciados por el rendimiento histórico de los equipos. Un equipo con mejor promedio ofensivo o que enfrente a un rival con mayor promedio de goles recibidos tenderá a tener un valor esperado más alto.

Después se calcula la posesión de balón a partir de una comparación basada en ranking FIFA. También se seleccionan 11 titulares por equipo. En la versión actual, la selección de titulares es simple y toma un conjunto fijo de jugadores; esto es suficiente para cumplir el requisito de registrar convocados, aunque podría ampliarse en versiones futuras.

Para cada jugador titular se simulan estadísticas individuales: goles, minutos jugados, tarjetas, faltas y asistencias. Los goles del equipo se reparten entre jugadores mediante eventos probabilísticos. Cuando el partido termina, se actualizan las estadísticas históricas del equipo y de los jugadores, además de las estadísticas específicas de la copa actual.

3.6 Resolución de empates en eliminación directa

En fase de grupos se permite el empate, porque ese resultado forma parte natural del sistema de puntos. Sin embargo, en fases eliminatorias debe existir un ganador. Cuando un partido de eliminación directa termina empatado, el sistema activa una resolución por prórroga.

La prórroga se modela agregando un gol al equipo ganador del desempate. La probabilidad de ganar se sesga con base en el ranking FIFA: un equipo mejor rankeado tiene mayor probabilidad de imponerse, aunque no se elimina por completo la posibilidad de que gane el equipo con peor ranking. Además, cuando hay prórroga, las estadísticas individuales del partido se recalculan considerando 120 minutos.

3.7 Construcción de tablas de grupos

Cada grupo construye una tabla con los cuatro equipos participantes. Para cada selección se acumulan puntos, diferencia de goles y goles a favor. La clasificación se ordena utilizando los criterios principales:

1. Mayor cantidad de puntos.
2. Mayor diferencia de goles.
3. Mayor cantidad de goles a favor.
4. Criterio aleatorio de desempate cuando persiste la igualdad.

Esta tabla permite obtener el primero, segundo y tercero de cada grupo. Los terceros son luego comparados entre todos los grupos para elegir los ocho mejores clasificados.

3.8 Transición a R16

La transición a R16 se implementó como un procedimiento de selección y validación. Primero se obtienen los 12 primeros de grupo, los 12 segundos y los 12 terceros. Luego se ordenan los terceros y clasifican los ocho mejores.

El armado de cruces evita enfrentar equipos del mismo grupo. Además, se valida que existan exactamente 16 partidos, que se usen 32 equipos distintos y que ningún equipo aparezca más de una vez. Esta validación es importante porque permite detectar errores antes de simular la fase eliminatoria.

La estructura de cruces se divide en bloques:

- Parte de los primeros de grupo se enfrenta contra terceros clasificados.
- El resto de primeros se enfrenta contra segundos de menor rendimiento.
- Los segundos restantes se enfrentan entre sí.

Este diseño sigue una lógica más elaborada que simplemente emparejar equipos al azar.

3.9 Fases eliminatorias posteriores

Después de R16, el sistema genera las demás fases con base en los ganadores de la ronda anterior. Los ganadores de R16 pasan a R8, los ganadores de R8 pasan a cuartos de final, los ganadores de cuartos pasan a semifinales, y de las semifinales salen los finalistas y los equipos que disputan el tercer puesto.

Cada fase se genera solo después de que la fase anterior haya sido simulada. Si se intenta generar una ronda sin ganadores válidos, el sistema lanza un error. Esto evita avanzar con estados incompletos.

3.10 Reporte final

El reporte final presenta los resultados más importantes del torneo:

- campeón;
- subcampeón;

- tercer puesto;
- cuarto puesto;
- máximo goleador del equipo campeón;
- tres mayores goleadores de la copa;
- equipo con más goles históricos actualizados;
- confederación con mayor presencia en R16, R8 y R4.

Una mejora importante de la versión final fue separar estadísticas históricas y estadísticas de la copa actual. Así, los goleadores del reporte se calculan usando únicamente los goles marcados durante esta edición simulada, no los goles acumulados históricamente.

3.11 Persistencia de datos actualizados

Al finalizar el torneo, el sistema genera dos archivos CSV:

- `equipos_actualizados.csv`
- `jugadores_actualizados.csv`

El archivo de equipos almacena los históricos actualizados de cada selección. El archivo de jugadores almacena tanto estadísticas históricas como estadísticas de la copa actual. Esta separación permite conservar el estado actualizado del torneo y facilita revisar los resultados desde una herramienta externa como Excel.

3.12 Medición de recursos

El sistema incluye un medidor de recursos que cuenta iteraciones y estima memoria. Cada fase del torneo reinicia el contador y luego reporta sus resultados. Esto permite comparar el costo relativo de operaciones como carga de equipos, conformación de grupos, generación de calendario, simulación de grupos, transición a R16 y persistencia.

La memoria se estima con base en el tamaño de las estructuras principales. No se trata de una medición exacta del sistema operativo, sino de una aproximación útil para documentar el consumo de memoria de las estructuras del programa

4. Problemas de desarrollo que afrontó

Durante el desarrollo se presentaron varios problemas técnicos y de diseño.

El primer problema importante fue la lectura del archivo CSV. Inicialmente el programa no encontraba el archivo, debido a confusiones con la ruta de ejecución de Qt Creator. Se corrigió usando una ruta absoluta al archivo de entrada y verificando que el programa realmente tuviera acceso al CSV.

También aparecieron errores de enlace, como referencias indefinidas a métodos de `Jugador`. Esto ocurrió porque algunas funciones estaban declaradas en el archivo `.h`, pero no implementadas en el `.cpp`, o porque las firmas no coincidían exactamente. El problema se solucionó revisando la correspondencia entre declaraciones e implementaciones.

Otro problema fue la conformación de grupos. Una asignación directa podía producir configuraciones inválidas por restricciones de confederación. Para resolverlo, se reemplazó la lógica inicial por un algoritmo de backtracking que prueba asignaciones, retrocede cuando encuentra una configuración inválida y continúa hasta lograr grupos válidos.

La transición a R16 también requirió ajustes. Era necesario garantizar que clasificaran los equipos correctos y que los cruces no repitieran equipos ni enfrentaran selecciones del mismo grupo. Para ello se agregó una función de validación de R16 y una impresión de clasificados, lo que permitió auditar visualmente los resultados.

La calendarización de grupos fue otro punto que evolucionó. La primera solución intentaba ubicar partidos con una estrategia greedy, pero la salida resultaba difícil de leer y podía ser menos robusta. Finalmente se adoptó una calendarización por jornadas, mucho más clara y eficiente.

También surgieron problemas de consistencia entre `Partido.cpp` y `Equipo`. Algunas funciones esperadas por `Partido`, como `seleccionarTitulares`, `actualizarHistoricoEquipo` y `actualizarJugadoresConvocados`, no existían o tenían firmas distintas en `Equipo`. La solución fue alinear las interfaces entre los archivos `.h` y `.cpp`, asegurando que las funciones declaradas coincidieran exactamente con las llamadas reales.

Finalmente, el reporte final mezclaba inicialmente estadísticas históricas con estadísticas de la copa actual. Esto podía producir resultados ambiguos en la sección de goleadores. Para solucionarlo se agregaron campos específicos para la copa actual dentro de `Jugador`, y el reporte final se modificó para usar esos valores.

5. Evolución de la solución y consideraciones a tener en cuenta en la implementación

La solución evolucionó de una estructura inicial más simple hacia una arquitectura más robusta y modular. Al comienzo, el objetivo principal era lograr que el programa cargara los equipos, generara grupos y simulara resultados. Sin embargo, a medida que se revisaron los requisitos, se identificó que el desafío exigía mayor control: restricciones de confederación, calendario realista, actualización de históricos, reporte final completo, persistencia y medición de recursos.

Una de las primeras decisiones importantes fue dividir el proyecto en clases especializadas. Esta separación permitió que cada archivo tuviera una responsabilidad clara. `Torneo` coordina el flujo principal, `Grupo` gestiona la fase de grupos, `Partido` simula encuentros, `Equipo` almacena datos de selección, `Jugador` almacena estadísticas individuales, `Utilidades` concentra funciones auxiliares y `MedidorRecursos` registra métricas.

La conformación de grupos evolucionó hacia una solución con backtracking, porque una asignación lineal no garantizaba cumplir las restricciones. Esta mejora hizo que el sorteo fuera más confiable y defendible desde el punto de vista algorítmico.

La calendarización también cambió considerablemente. La versión final por jornadas es más eficiente y más fácil de verificar. Además, la impresión del calendario se mejoró para mostrar fecha, grupo y jornada, lo cual hace que la salida sea más clara para el usuario y para quien evalúe el proyecto.

La simulación de partidos se fortaleció al corregir la fórmula de goles esperados. La implementación final usa el promedio histórico de goles a favor del equipo atacante y el promedio histórico de goles en contra del rival. Posteriormente, los goles esperados se convierten en goles reales mediante una distribución probabilística, evitando que los partidos sean completamente deterministas.

Otra mejora clave fue separar las estadísticas históricas de las estadísticas de la copa actual. Esto permitió que el reporte final fuera más coherente, especialmente en las secciones de goleadores. Además, el archivo `jugadores_actualizados.csv` guarda ambos tipos de información, lo que facilita analizar tanto el acumulado histórico como el desempeño específico del torneo simulado.

Desde el punto de vista de memoria, se procuró evitar asignaciones dinámicas innecesarias. Se usaron `std::array` cuando el tamaño era fijo, como en los 26 jugadores de cada equipo, los 11 titulares, los seis partidos de cada grupo y los 12 grupos. Se usaron `std::vector` solo cuando la cantidad de elementos podía crecer o cuando era conveniente manejar colecciones variables, como bombos, equipos clasificados y partidos de fases eliminatorias.

Para la implementación final es importante tener en cuenta varias consideraciones. Primero, el archivo CSV de entrada debe encontrarse en la ruta definida en `main.cpp`; de lo contrario, el programa no podrá iniciar. Segundo, la salida detallada de partidos puede ser muy extensa, por lo que conviene manejarla mediante una bandera de control. Tercero, los archivos de salida se generan en el directorio de ejecución del programa, no necesariamente en la carpeta donde están los archivos fuente. Cuarto, la memoria reportada por el sistema corresponde a una estimación estructural, no a una medición exacta del sistema operativo.

En conclusión, la solución final no se limita a simular resultados de forma aleatoria, sino que integra lectura de datos, modelado orientado a objetos, algoritmos de asignación con restricciones, simulación probabilística, actualización de históricos, persistencia y generación de reportes. Esto permite presentar una solución completa, modular y justificable para el desafío planteado.